

TP — Mini Moteur MapReduce : MPI vs Apache Spark

Master IA - FSBM

Réalisé par : BOUSSAID Salama et CHAABAN MALIKA

Année universitaire : 2025/2026

Objectif du TP :

L'objectif de ce TP est d'implémenter from scratch un mini moteur MapReduce en Python avec MPI (mpi4py), en codant manuellement les trois phases fondamentales — Map, Shuffle et Reduce — afin de comprendre concrètement comment fonctionne le traitement parallèle distribué. Le TP vise ensuite à comparer cette implémentation avec Apache Spark, un framework industriel qui automatise ces mêmes phases, à travers des benchmarks de performance sur un problème de WordCount, pour analyser les compromis entre contrôle bas niveau (MPI) et simplicité d'utilisation (Spark).

1. Installation de WSL2 (Ubuntu sur Windows)

Ouvrez PowerShell en tant qu'administrateur et exécutez la commande suivante :

```
wsl --install
```

Si vous rencontrez une erreur réseau, installez Ubuntu via le Microsoft Store :

- Ouvrez Microsoft Store
- Recherchez Ubuntu 24.04
- Cliquez sur Installer puis Ouvrir
- Créez votre nom d'utilisateur et mot de passe Linux

 *Quand vous tapez le mot de passe, rien ne s'affiche — c'est normal !*

2. Mise à jour du système Ubuntu

Dans le terminal Ubuntu, exécutez :

```
sudo apt update
```

3. Installation de Python, pip et les outils système

Installez Python3, pip et MPICH :

```
sudo apt install -y python3 python3-pip mpich
```

4. Remplacer MPICH par OpenMPI

 *MPICH ne fonctionne pas bien sur WSL. Il faut utiliser OpenMPI.*

Exécutez les commandes suivantes :

```
sudo apt remove mpich -y  
sudo apt install openmpi-bin libopenmpi-dev -y
```

5. Installation de Java (requis pour Spark)

Installez le JDK par défaut :

```
sudo apt install default-jdk -y
```

Vérifiez l'installation :

```
java -version
```

6. Aller dans le dossier du projet

Cloner le projet a partir de git hub, le lien : [SalamaBsd/Projet4_mapreduce MPi_spark](https://github.com/SalamaBsd/Projet4_mapreduce_MPi_spark)
puis :

```
cd "/mnt/c/Users/TonNom/Desktop/P4_mapreduce_project"
```

⚠ Remplacez *TonNom* par votre nom d'utilisateur Windows.

7. Créer l'environnement virtuel Python

```
python3 -m venv venv  
source venv/bin/activate
```

⚠ Vous devez voir (venv) apparaître devant votre terminal.

8. Installer les dépendances Python

Installez les packages nécessaires :

```
pip install mpi4py pyspark  
pip install matplotlib --timeout 120  
pip install jupyter pandas --timeout 120
```

Vérifiez les installations :

```
python -c "from mpi4py import MPI; print('MPI OK')"  
python -c "import pyspark; print('Spark OK')"
```

9. Créer un lien symbolique pour MPI

⚠ MPI ne fonctionne pas depuis /mnt/c/ (chemin Windows avec espaces). On crée un raccourci dans le home Linux.

```
ln -s "/mnt/c/Users/TonNom/Desktop/P4_mapreduce_project" ~/proj
cd ~/proj
source venv/bin/activate
```

10. Tester MPI

```
mpirun -n 4 python mpi/mapreduce.py data/big_input.txt
```

Résultat attendu (4 processus en parallèle) :

```
[MAP] Processus 0 : 4 mots uniques
[MAP] Processus 1 : 5 mots uniques
[MAP] Processus 2 : 4 mots uniques
[MAP] Processus 3 : 5 mots uniques
[SHUFFLE] ...
[REDUCE] ...
✓ TERMINÉ
```

11. Lancer Jupyter Notebook

```
cd ~/proj
source venv/bin/activate
jupyter notebook
```

- Ouvrez le lien <http://localhost:8888/> (vas etre dans résultats de execution de jupyter notebook ,copier tout la ligne)
- Cliquez sur [mapreduce_project.ipynb](#)
- Exécutez les cellules et fait la compariason entre MPI (mpi4py) et Apache Spark (PySpark) sur le problème WordCount.

12. Structure du projet

```
P4_mapreduce_project/
├── data/
│   ├── input.txt
│   └── big_input.txt
└── mpi/
```

```
├── mapreduce.py
├── benchmark.py
├── spark/
│   ├── wordcount_spark.py
│   └── benchmark_spark.py
├── results/
│   ├── bench_mpi.csv
│   ├── bench_spark.csv
│   ├── comparison.png
│   └── mapreduce_project.ipynb
```

14. Description de chaque fichier du projet :

- ✓ **data/input.txt** : Fichier texte de base (4 lignes) utilisé comme données d'entrée pour tester le WordCount.
- ✓ **data/big_input.txt** : Le même texte répété 100 fois pour avoir un volume suffisant pour les benchmarks.
- ✓ **mpi/mapreduce.py** : Implémente les 3 phases MapReduce avec MPI : chaque processus reçoit sa tranche de texte (Map), redistribue les mots entre processus via MPI_Send/Recv (Shuffle), puis agrège les comptages (Reduce).
- ✓ **mpi/benchmark.py** : Relance mapreduce.py avec 1, 2 puis 4 processus et mesure le temps d'exécution de chaque run, puis sauvegarde les résultats dans bench_mpi.csv.
- ✓ **spark/wordcount_spark.py** : Fait le même WordCount qu'avec MPI mais en PySpark en une chaîne de transformations : flatMap (Map) → reduceByKey (Shuffle + Reduce).
- ✓ **spark/benchmark_spark.py** : Exécute le WordCount Spark sur les deux fichiers (petit et grand) et mesure les temps, sauvegardés dans bench_spark.csv.
- ✓ **results/bench_mpi.csv** : CSV contenant les temps d'exécution MPI pour 1, 2 et 4 processus.
- ✓ **results/bench_spark.csv** : CSV contenant les temps d'exécution Spark pour le petit et le grand fichier.
- ✓ **results/comparison.png** : Graphiques générés automatiquement comparant visuellement les temps MPI vs Spark.

- ✓ **mapreduce_project.ipynb** : Notebook principal qui orchestre tout : appelle les scripts MPI et Spark via `subprocess`, lance les benchmarks, puis affiche les graphiques et le tableau comparatif final.

15. En cas de problème

15.1 pip timeout lors du téléchargement

```
pip install <package> --timeout 120
```

15.2 MPI affiche seulement Processus 0

```
sudo apt remove mpich -y  
sudo apt install openmpi-bin libopenmpi-dev -y  
pip install --force-reinstall mpi4py
```

15.3 Java non trouvé pour Spark

```
sudo apt install default-jdk -y
```

15.4 Erreur "No such file or directory" avec mpirun

⚠ *Le chemin contient des espaces — utilisez le lien symbolique.*

```
ln -s "/mnt/c/Users/TonNom/Desktop/P4_mapreduce_project" ~/proj  
cd ~/proj
```